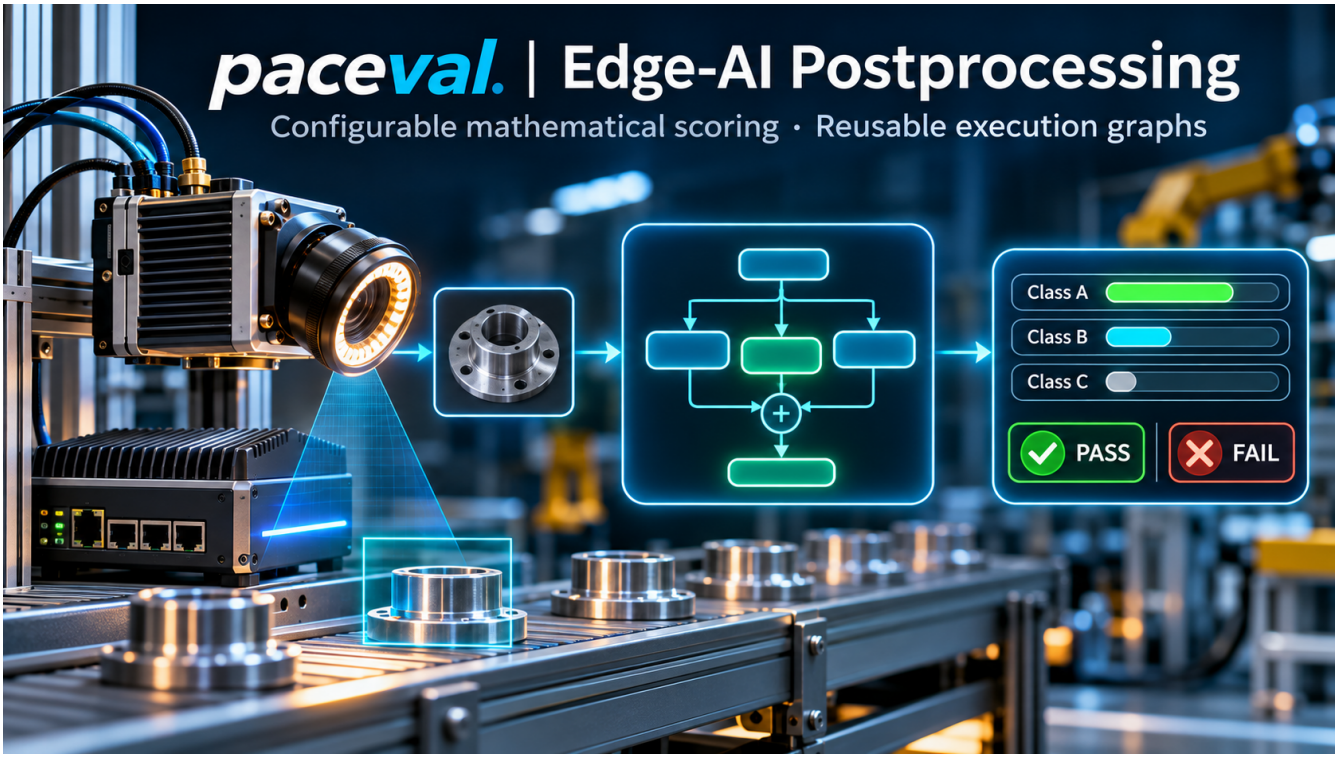
Programmable Mathematical Scoring for Edge-AI Postprocessing

Reusable execution graphs, audited formula versions, and a validated synthetic inference-output workflow



Scientific status: implemented and independently verified offline mathematical demonstrator. The project does not execute a neural network, use real industrial sensor measurements, certify a detector, or claim a performance improvement.

Evidence: 50 versioned scoring formulas; 50 corresponding threshold-margin expressions; two TINC-enabled graphs; 288 synthetic input records from three traces; 28,800 mathematically audited outputs; all 24 selected TINC pointwise enclosures containing the 110-digit reference result.

Publication artifact: README.md, src/, examples/, and figures/ in the same repository.

1. Scientific motivation and scope

The supplied research proposal identifies a testable component: a repeatedly evaluated postprocessing formula operating on model outputs. Its first-study design specifies three precomputed inference-output traces, 50 versioned scoring formulas, and repeated scalar or batched calls. The inference engine, data transfer, sampling, and any device control remain outside the paceval kernel.

This implementation advances that proposal from a proposed test to an executable numerical feasibility study. The input traces are intentionally synthetic; thus the conclusions are about mathematical execution and validation, not classification accuracy or on-device deployment.

External ecosystem context

ONNX Runtime supports deployment to mobile/edge platforms through its execution-provider architecture. OpenVINO describes model-output postprocessing, including customized operations. TensorFlow Lite detection examples describe confidence thresholds and result filtering. These references establish a real integration boundary: inference output data can enter a separate application-specific mathematical rule layer [1–3].

Specific research questions

Research question	Implementable test
Can expressions be preserved and reused?	Create persistent paceval handles and evaluate three fixed traces in batches.
Can formula changes be made in a controlled manner?	Read a version manifest; change one formula weight without editing the C++ evaluator.
Can downstream decision logic be kept separate?	Use paceval for score and margin, with EMA and hysteresis owned by the host.
Can numerical behavior be audited independently?	Compare every graph result to native C++ and 110-digit mpmath; audit selected TINC outputs.

The work does not replace non-maximum suppression, model inference, calibration-data collection, detector evaluation or operational approval. The threshold is part of the synthetic example, not a validated production limit.

2. Mathematical models and declared domain

The central score is a scalar, dimensionless illustrative quantity, not a calibrated probability:

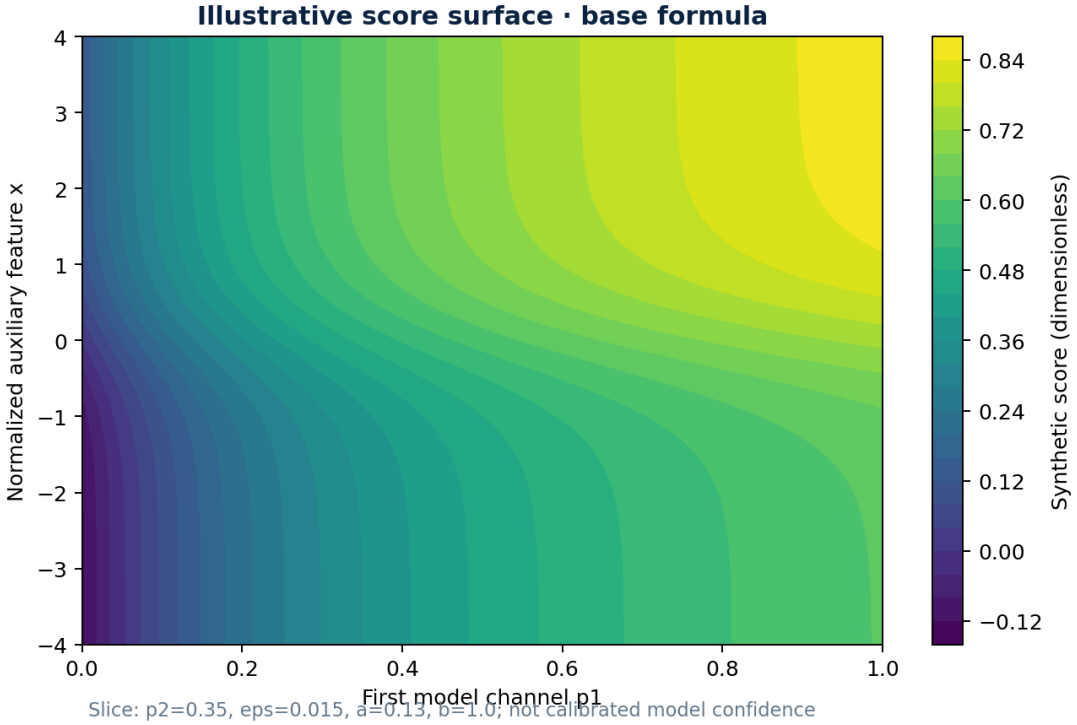
$$s_0 = p_1 / (p_1 + p_2 + \varepsilon) + a \cdot \tanh(b \cdot x)$$

The two channels p_1, p_2 are each in [0, 1], x is a normalized auxiliary feature in [−4, 4], quality is in [0, 1], a in [0, 0.3], b in [0.1, 2.5], and the denominator regularizer ε is in [0.001, 0.05]. Input validation rejects nonfinite values and domain violations before paceval is called.

Five explicitly defined families of controlled corrections

Family	Correction term multiplied by manifest weight w
0	quality − 0.5
1	$(p_1 - p_2) \cdot$ quality
2	$x^2 / (1 + x^2)$
3	$(p_1 - p_2)^2$
4	sin(x) · quality

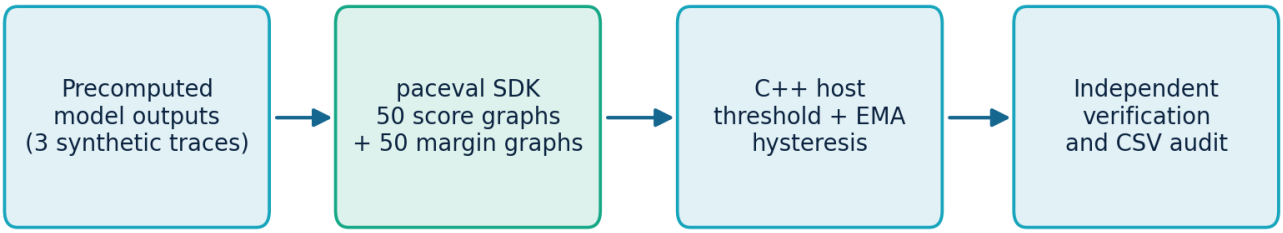
Each family has ten revisions, for 50 manifest entries total. The resulting function is $s = s_0 +$ w·correction. The coefficient w lies in [0, 0.1]. The separate margin expression is m = s − threshold. Revision 0 in family 0 has w = 0 and reproduces the exact base formula from the source proposal.



Slice: p2=0.35, eps=0.015, a=0.13, b=1.0; not calibrated model confidence

3. Implementation architecture

Validated edge postprocessing boundary



No neural-network inference, device I/O, or hardware control is performed by this demonstrator

paceval responsibility: create and retain 50 score graphs and 50 corresponding threshold-margin graphs; evaluate them through the documented multi-computation batch API; create two additional interval-enabled graphs for selected numerical checks.

Host responsibility: load and validate manifest and trace CSV files; enforce the input envelope; calculate the independent native C++ reference; maintain per-trace and per-version smoothing and hysteresis state; export results and provenance.

Verified SDK calls

API function	Demonstrated role
paceval_InitializeLibrary / paceval_FreeLibrary	Initialize and close the runtime in an exception-safe RAII class.
paceval_CreateComputation / paceval_DeleteComputation	Create and release persistent, versioned expression handles.
paceval_dGetMultipleComputationsResultsExt	Evaluate a common sample-major input array across multiple graphs; read graph-major outputs.
paceval_dGetComputationResult	Read pointwise TINC-enabled value, lower and upper endpoints.
paceval_GetIsError / paceval_GetErrorInformation	Check analysis and computation results explicitly.

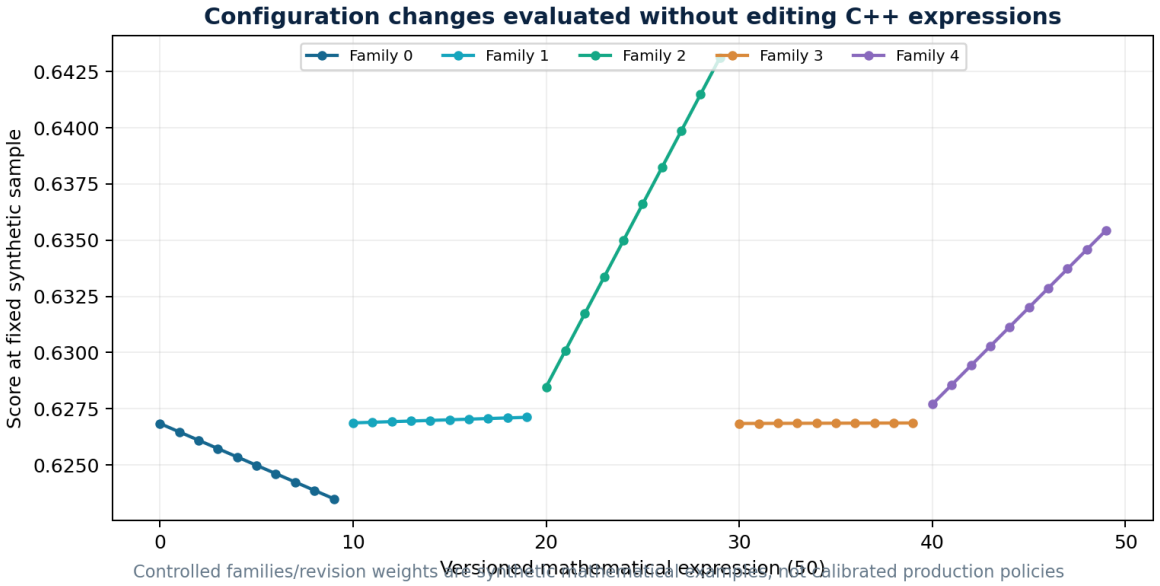
The SDK facade files src/paceval_main.h and src/paceval_main.cpp are included as original API source, preserving their AGPL notices. The separately provided paceval Linux runtime is not distributed.

4. Trace orchestration and audited formula revisions

Three illustrative traces with 96 rows each are labeled vision, vibration and thermal. All contain the same two numeric model-output channels and the same auxiliary feature schema, with different deterministic trajectories. They represent already computed hypothetical outputs, not sensor measurements or trained-network inference.

Version registry: examples/formulas.csv

The manifest contains version_id, family, revision and weight. The program constructs the expression from the audited template family and the declared coefficient. This is controlled configurability: the host does not accept arbitrary unmanaged expression syntax. For change control, examples/formulas_updated.csv changes one version identifier and its coefficient while leaving the other 49 unchanged.



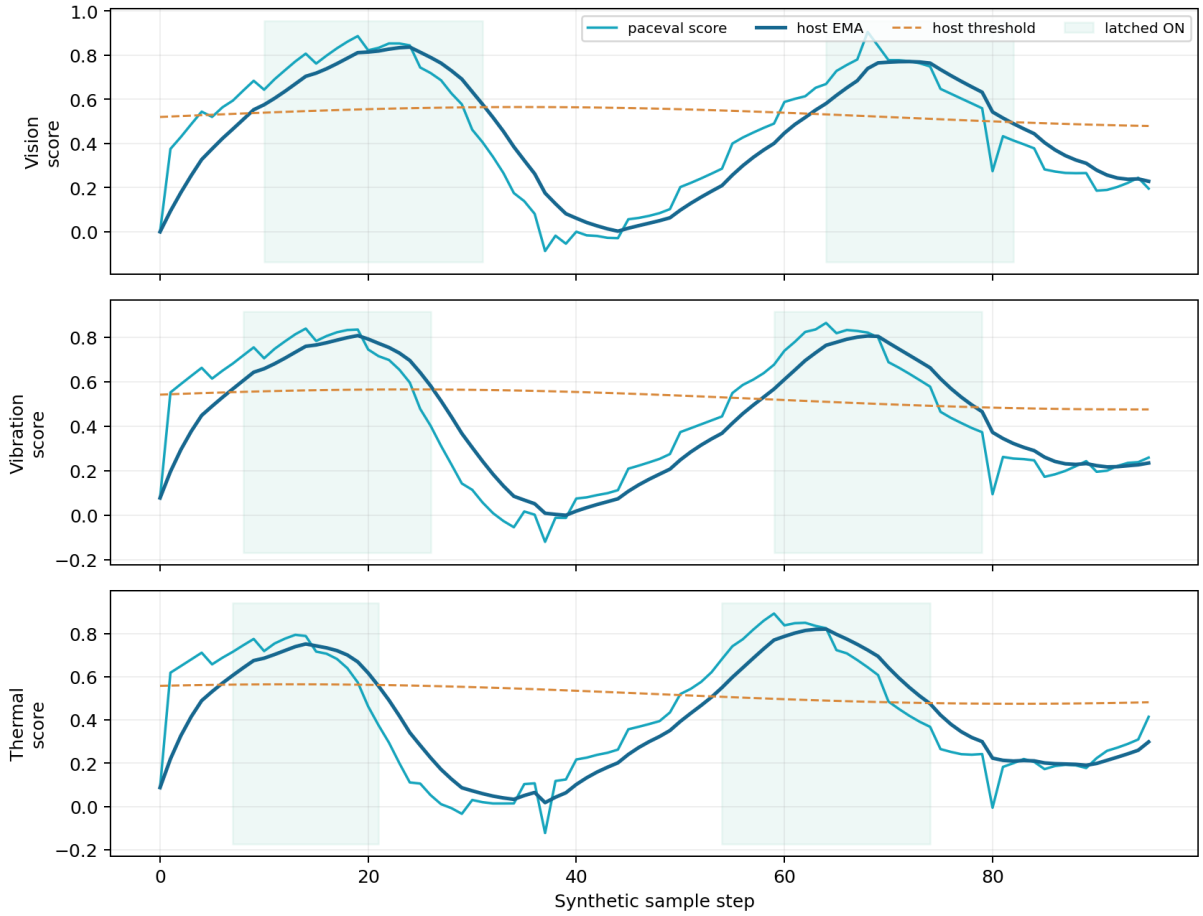
Host-side decision sequence

For each (trace, version) pair, a new score initializes the exponential moving average. Subsequent values use $EMA_n = 0.25 \cdot score_n + 0.75 \cdot EMA_{n-1}$. Latching is switched ON above threshold + 0.025 and OFF below threshold − 0.025, retaining the previous state in between. These are illustrative deterministic rules, not safety-critical control.

5. Temporal decision output across three synthetic streams

Three independent, temporally ordered input streams are evaluated under the same versioned mathematical rule. The score and threshold margin come from paceval, while the temporal EMA and hysteresis state are deliberately maintained outside the execution graphs in C++. The figure presents only synthetic, uncalibrated mathematical signals.

Scoring, host-owned smoothing and threshold decisions



Because each formula version has its own state for each trace, switching the active formula cannot silently reuse the decision state of a different version. The complete CSV stores trace ID, sample step, formula version, numerical outputs, the EMA and the final illustrative latched state.

6. Independent verification and observed result

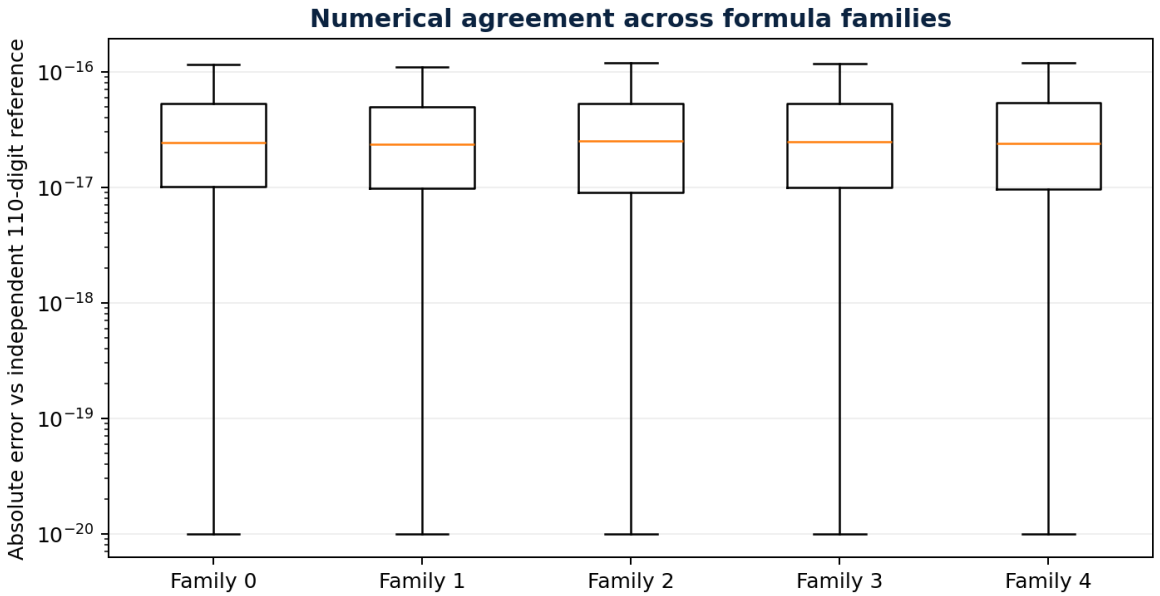
Scope: 288 input records × 50 formula versions × two mathematical outputs = 28,800 scalar evaluations. Full sequences are evaluated with pre-created graphs and multiple batch sizes. All 28,800 score/margin results passed comparison to a separately coded native C++ expression.

High-precision audit: src/validate_outputs.py independently recomputes all 28,800 mathematical outputs at 110 decimal digits using mpmath, audits the host EMA/latching state, and checks TINC interval containment. Across the audited dataset, the maximum absolute difference between recorded binary64 scores/margins and the 110-digit value was approximately 2.06×10^{-16}, with zero tolerance failures.

TINC: 24 selected interval-enabled evaluations (score or margin) enclosed their independently computed 110-digit reference value: 24/24. This is an empirical pointwise result, not a proof of global enclosure or a formal certificate.

Configuration-regression test: changing one manifest entry changed exactly 288 score rows for that version and did not alter the other 49 version outputs.

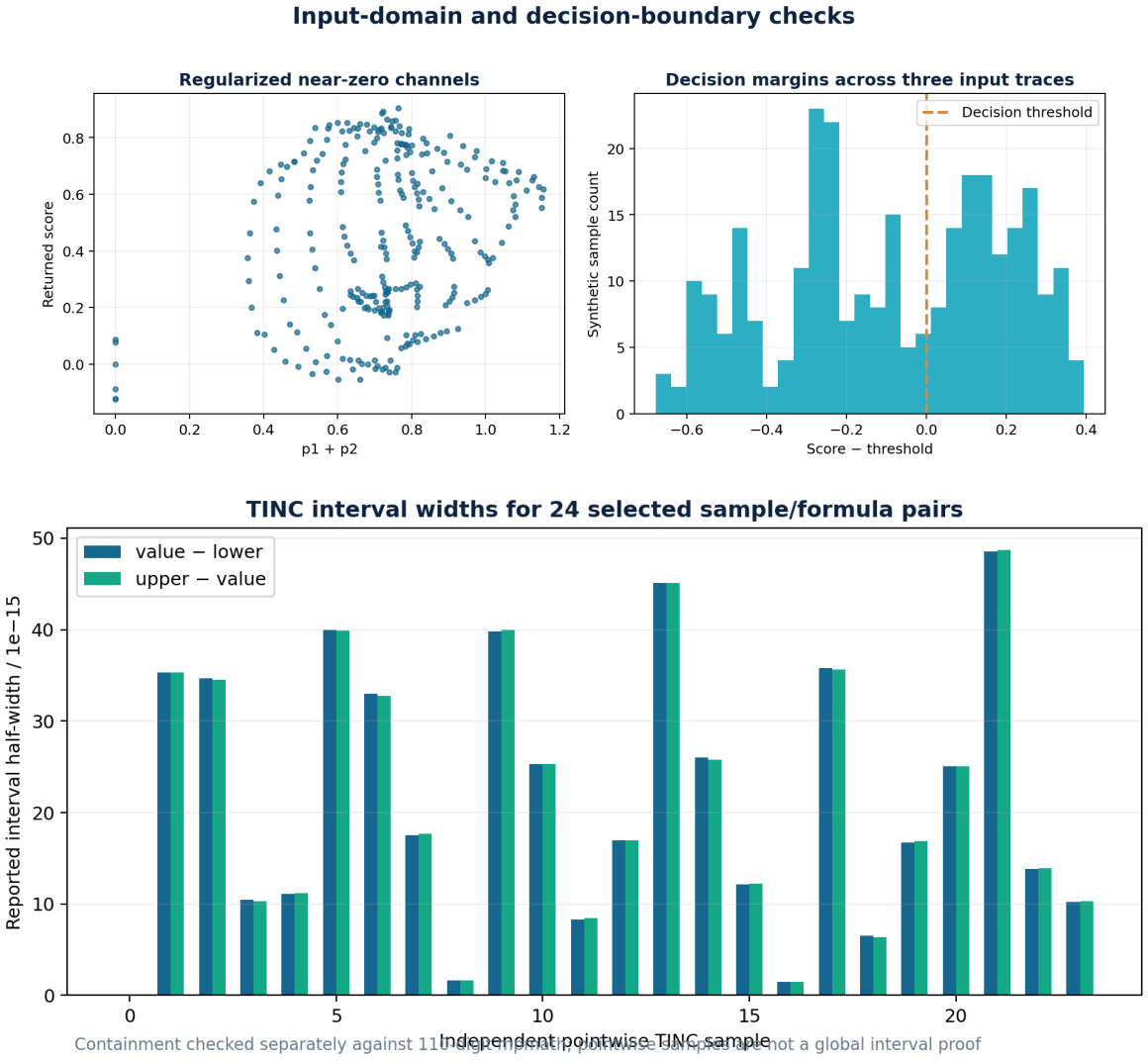
Determinism and build checks: outputs from batch sizes 37 and 64 were byte-identical in this tested environment. GCC 14.2, Clang 17, AddressSanitizer and UndefinedBehaviorSanitizer tests passed. A nonfinite or malformed input is rejected before execution.



Numerical tolerances and the full reference procedure are defined in src/edge_ai.cpp and src/validate_outputs.py. Changes to compiler math semantics, runtime build, input precision or expression definitions require renewed verification.

7. Boundary cases and pointwise interval investigation

The examples include rows with $p_1 = p_2 = 0$, normalized auxiliary features at $x = +4$ and $x = -4$, and threshold margins on both sides of zero. The strictly positive ε prevents division by zero for the defined domain. Nonfinite and out-of-domain inputs are rejected.



The chart depicts reported interval widths. The independent audit tests reference containment using exact binary64 input values lifted into 110-digit arithmetic. Finite-sample inclusion does not imply a guarantee for arbitrary model inputs or expression families.

8. Reproducible build and research artifacts

The project includes only the needed application source, paceval SDK facade, input/output CSV records, report, and reproducible diagrams. The mathematical runtime remains a separate externally supplied binary. The main README contains full instructions.

Linux build with direct SDK integration

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release
-DPACEVAL_RUNTIME=/absolute/path/libpaceval_linux_sharedLIB.so

cmake --build build -j
ctest --test-dir build --output-on-failure

./build/edge_ai --catalog examples/formulas.csv
--traces examples --limit 96 --batch 64
--output results/scores.csv --tinc results/tinc_samples.csv

python3 src/validate_outputs.py --scores results/scores.csv
--tinc results/tinc_samples.csv
```

File provenance and audit entry points

Repository file	Reproducibility purpose
src/edge_ai.cpp	C++ implementation, formula template validation and runtime audit.
src/paceval_main.h and src/paceval_main.cpp	Original official SDK interface layer with retained notices.
src/CMakeLists.txt	Direct linkage against externally provided Linux paceval runtime.
examples/formulas.csv and examples/formulas_updated.csv	Fifty versioned rules and a controlled one-rule revision.
examples/trace_vision.csv, trace_vibration.csv, trace_thermal.csv	Three synthetic precomputed input streams.
examples/scores.csv and examples/tinc_samples.csv	Recorded results with provenance and numerical interval information.
src/validate_outputs.py and src/make_figures.py	Independent 110-digit validation and publication-figure reproduction.

Tested shared runtime SHA-256: acf63a87df0ee822b467edbca887393d31ec0ea03562e1abf802d2512a5bfb12

9. Scientific interpretation and next steps

The defined application demonstrates that paceval can successfully evaluate and reuse a controlled family of mathematical postprocessing rules after a hypothetical inference stage, including configurable versions, threshold margins, batched input evaluation and selected interval diagnostics. Independence of the native and high-precision audits gives this finite experiment a reproducible mathematical basis.

A practical collaboration would next replace the synthetic channels with a pinned inference model and approved test records, define model-calibrated thresholds, establish domain semantics for missing or anomalous data, and evaluate the complete pipeline against task-specific correctness criteria. Any embedded latency, memory, energy, or FPGA/ASIC claims would require separate measurements.

Selected authoritative resources

[1] OpenVINO 2026. Pre- and post-processing interface reference.

https://docs.openvino.ai/2026/openvino-workflow/running-inference/optimize-inference/optimize-preprocessing/preprocessing-api-details.html

[2] ONNX Runtime. Deploy on mobile / edge devices.

https://onnxruntime.ai/docs/tutorials/mobile/

[3] TensorFlow Lite. Object detection model outputs and thresholds.

https://github.com/tensorflow/tensorflow/blob/master/tensorflow/lite/g3doc/examples/object_detection/overview.md

[4] paceval. Official C/C++ mathematical API.

https://paceval.com/api/

[5] NIST. AI Risk Management Framework.

https://www.nist.gov/itl/ai-risk-management-framework

Publication statement: This is an independent offline numerical demonstration by paceval UG, not a reported customer integration, product certification, or general performance claim. All model-output traces, rule families and decision thresholds are synthetic. The generated hero image is illustrative artwork.

Contact: info@paceval.com | https://paceval.com